

DSC 40B Discussion 8 Worksheet Solutions

Friday, May 22, 2026

Imagine you are a tutor, and you need to explain something to someone who has only a vague understanding of what's going on. Do your best to get the point across. **ALTERNATE**.

If you're not sure, ask your peer if they know how to explain it, then REPEAT the explanation back. Remember, the point of the exercise is to **vocalize** your thoughts.

If you are both unsure, make sure to come to/tutor for help! Do not use ChatGPT or any other LLM! The point of discussion is to prepare you for the exams.

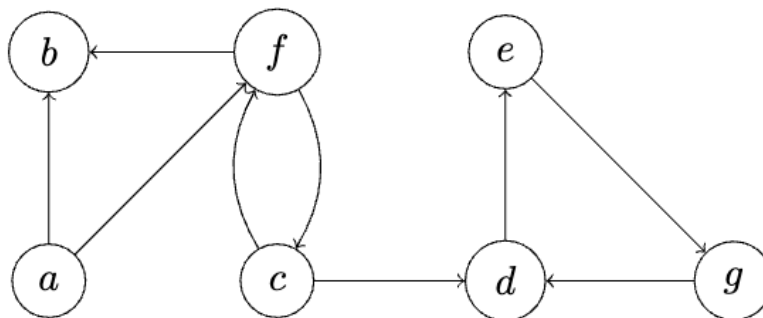
Have one person in the pair open the Discussion 1 Gradescope assignment. After both you **and** your partner finish a problem **and explain it**, answer the corresponding multiple choice/short answer questions for both versions.. You will receive instant feedback on the questions, whether you are correct or not. If your answer is incorrect, please go back to the question to understand it and feel free to ask tutors about anything you are confused about. Once you have finished the entire worksheet or discussion is ending, turn in your assignment and add your partner to the submission.

Quick Icebreaker

Introduce yourself to your partner by sharing name, year, major, etc. If you could be anywhere right now instead of 40B discussion, where would you be? (Do not spend more than 2 minutes!)

Questions

Question 1:



First Person:

Consider a depth-first search (DFS) starting from **node c**, which nodes are visited and in what order? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

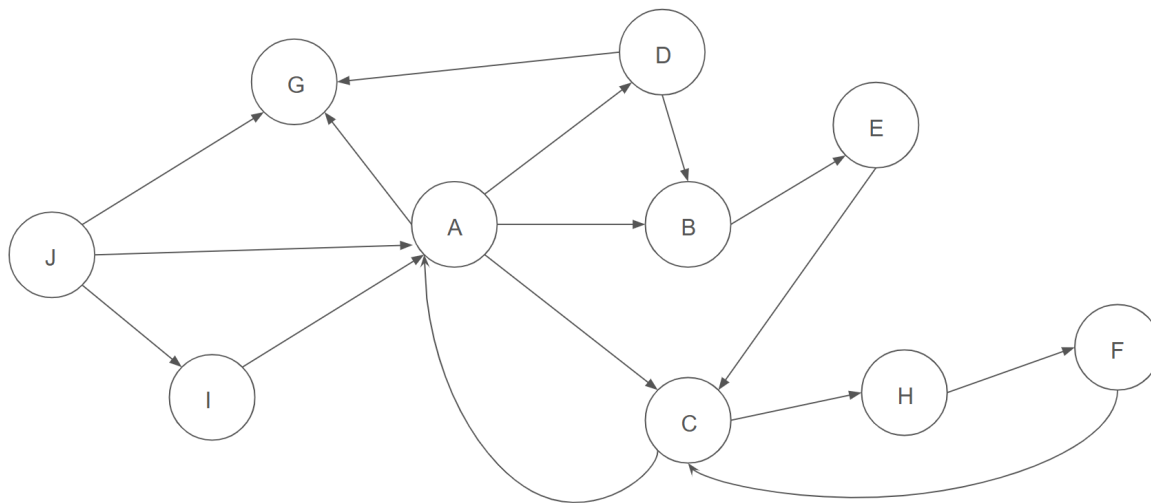
[g,e,d,b,f,c] is the sequence in which nodes are marked as visited. Node a is never discovered.

Second Person:

Consider a depth-first search (BFS) starting from **node a**, which nodes are visited and in what order? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

[b,g,e,d,c,f,a] is the sequence in which nodes are marked as visited.

Question 2:



Second Person:

Given the graph above and running DFS from the starting point of **node J**, what is the most amount of nodes in “pending” at the same time?

At the first moment where there were the most amount of nodes in “pending”, which nodes were visited and undiscovered at that time?

Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

The most nodes in the "pending" queue happens when [J,A,B,E,C,H,F] is the pending queue. No other nodes have been discovered or visited.

First Person:

Given the graph above and running DFS from the starting point of **node E**, what is the most amount of nodes in “pending” at the same time?

At the moment where there were the most amount of nodes in “pending”, which nodes were “visited” and “undiscovered” at that time?

Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

The most nodes in the "pending" queue happens when [E,C,A,D,G] is the pending queue. DFS had already discovered and marked B as visited. [J,I,H,F] are yet to be discovered at all.

Question 3:

First Person:

Given the graph in Question 2 and running DFS from the starting point of **node E**, what is the order that nodes are marked as visited? Use the convention that `graph.neighbors()` returns successors in **alphabetical order**. Explain your reasoning.

The overall sequence of nodes visited is [B,G,D,A,F,H,C,E].

Second Person:

Given the graph in Question 2 and running DFS from the starting point of **node J**, what is the order that nodes are marked as visited? Use the convention that `graph.neighbors()` returns successors in **reverse alphabetical order**. Explain your reasoning.

The overall sequence of nodes visited is [G,F,H,C,E,B,D,A,I,J].

Question 4:

Second Person:

In a graph search, what is the main conceptual difference between BFS and DFS? Explain your reasoning.

- ☐ A. BFS only works on undirected graphs, while DFS only works on directed graphs
- ☒ B. BFS processes the oldest pending node, while DFS processes the newest pending node
- ☐ C. BFS visits every node exactly once, while DFS can visit nodes multiple times
- ☐ D. BFS uses recursion, while DFS cannot use recursion

The correct answer is B because BFS processes the oldest pending node first, like a queue, while DFS processes the newest pending node first, like a stack. This is why BFS spreads outward level by level, while DFS follows one path deeply before backtracking.

First Person:

Suppose we run a graph search starting at node s . The search has already discovered nodes a , b , and c , in that order, and all three are currently pending. Assume no other pending nodes exist.

Which statement best describes what happens next? Explain your reasoning.

- ☒ A. ~~BFS processes a next, while DFS processes c next~~
- ☐ B. BFS processes c next, while DFS processes a next
- ☐ C. Both BFS and DFS process a next because it was discovered first
- ☐ D. Both BFS and DFS process c next because it was discovered most recently

The correct answer is A because a was discovered first, so BFS processes a next, while c was discovered most recently, so DFS processes c next. This shows the queue-like behavior of BFS versus the stack-like behavior of DFS.

Question 5:

First Person:

True or False. When running DFS on a directed graph, a node will only be marked as “visited”, when all of its neighbors/successors have been marked as “visited”. Explain your reasoning.

- ☐ True
- ☒ False

In DFS, a node is marked “visited” after DFS has finished exploring its outgoing edges, but not necessarily after all successors are marked visited, since some successors could already be pending in a cycle/back-edge situation. A more accurate statement would be that a node is marked visited after all of its undiscovered successors have been fully explored.

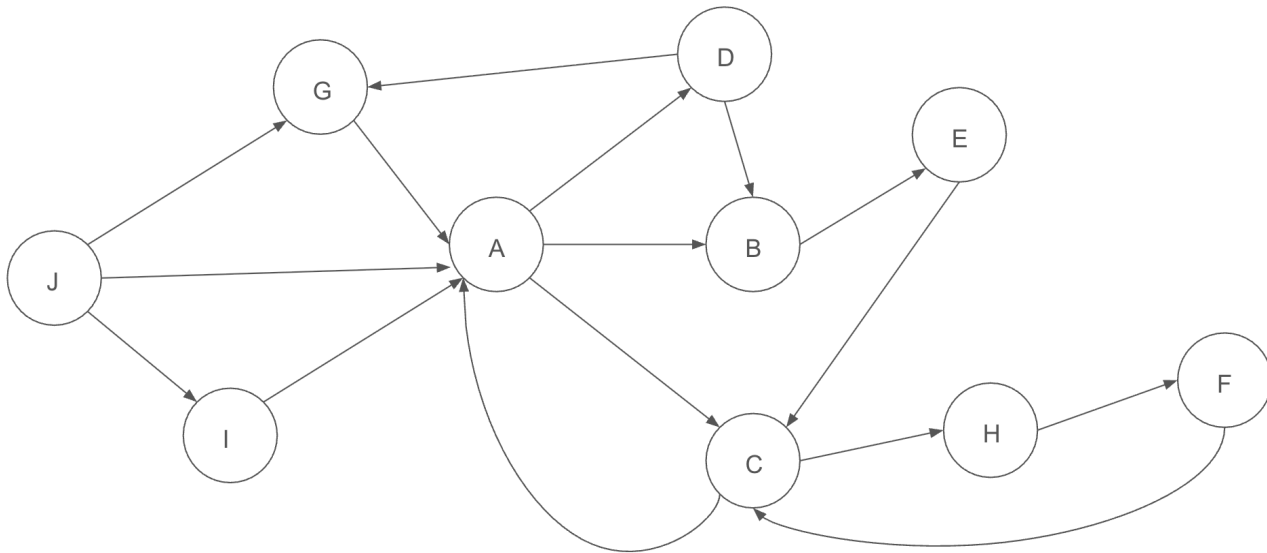
Second Person:

True or False. When running BFS on a directed graph, a node will only be marked as “visited”, when all of its neighbors/successors are marked as “pending”. Explain your reasoning.

- ☐ True
- ☒ False

In BFS, when a node is processed, its undiscovered successors may be marked pending, but successors that were already visited or already pending do not need to become pending again. So it is too strong to say all successors must be marked pending before the node becomes visited.

Question 6:



Second Person:

Given the graph above, run DFS starting from **node J**. Use the convention that `graph.neighbors()` returns successors in **alphabetical order**. How many back edges are found?

The 3 back edges found are [(C,A),(F,C),(G,A)].

First Person:

Given the graph above, run DFS starting from **node E**. Use the convention that `graph.neighbors()` returns successors in **alphabetical order**. How many back edges are found?

The 4 back edges found are [(B,E),(A,C),(G,A),(F,C)].

Both:

In general for any given directed graph, does the number of back edges discovered match the number of cycles in the graph? Explain your reasoning.

☐ Yes

☒ No

No, the number of back edges discovered is not necessarily the total number of cycles in the graph, as highlighted in the question above. It can vary with your starting node.

Question 7:

First Person:

Given the graph in Question 6, what is the minimum number of edges that can be removed to make it a DAG. List these edges and explain your reasoning.

The 3 edges to be removed are one of [(G,A), (A, D), (D, G)], (C,A), and one of [(F,C), (C, H), (H, F)].

Second Person:

Given the graph in Question 6, what is the minimum number of nodes that can be removed to make it a DAG. Removing a node also removes all edges that have the node as either a source or destination. List these nodes and explain your reasoning.

The 2 nodes to be removed are nodes C and one of A, D, or G.

Question 8:

Second Person:

Given the graph in Question 6 and running DFS from the starting point of **node C**, what are the edges that are in the resulting DFS tree? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

The edges in the resulting DFS tree, added in this order, are [(C,A),(A,B),(B,E),(A,D),(D,G),(C,H),(H,F)].

First Person:

Given the graph in Question 6 and running DFS from the starting point of **node I**, what are the edges that are in the resulting DFS tree? Use the convention that `graph.neighbors()` returns successors in alphabetical order. Explain your reasoning.

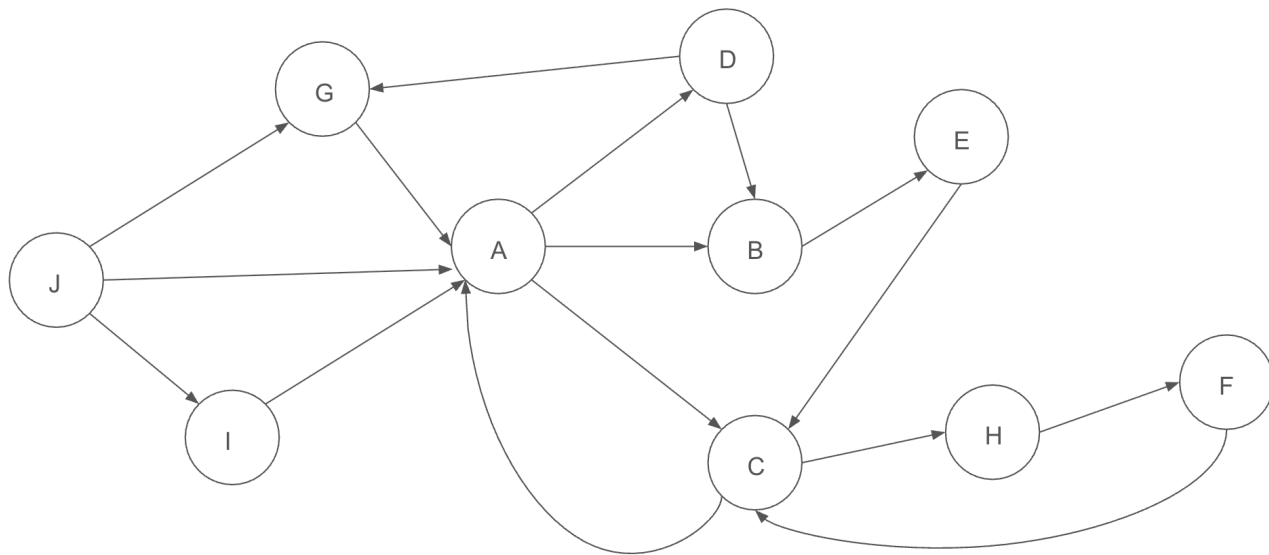
The edges in the resulting DFS tree, added in this order, are [(I,A),(A,B),(B,E),(E,C),(C,H),(H,F),(A,D),(D,G)]

Both:

Does the order of `graph.neighbors()` matter for the construction of the DFS tree?

Yes.

Question 9:



First Person:

Given the graph above, run full DFS and list the start and finish times of each node. Use the convention that `graph.neighbors()` and `graph.nodes` return successors/nodes in alphabetical order. Explain your reasoning.

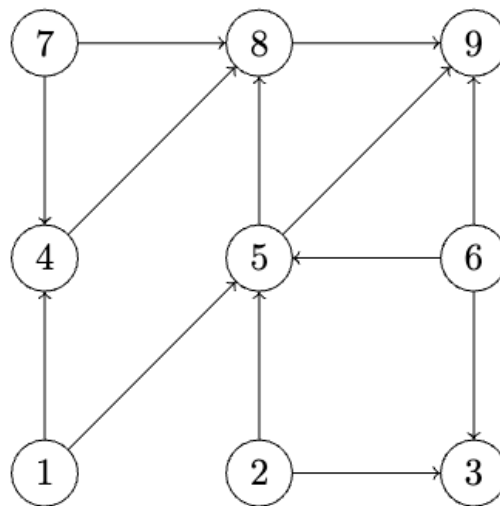
A: 1, 16
 B: 2, 11
 C: 4, 9
 D: 12, 15
 E: 3, 10
 F: 6, 7
 G: 13, 14
 H: 5, 8
 I: 17, 18
 J: 19, 20

Second Person:

Given the graph above, run full DFS and list the start and finish times of each node. Use the convention that `graph.neighbors()` and `graph.nodes` return successors/nodes in reverse alphabetical order. Explain your reasoning.

A: 3, 18
B: 7, 16
C: 9, 14
D: 4, 17
E: 8, 15
F: 11, 12
G: 5, 6
H: 10, 13
I: 2, 19
J: 1, 20

Question 10:



Second Person:

Given the directed graph above, is the following sequence a valid topological ordering? Explain your reasoning.

1, 2, 6, 7, 4, 5, 9, 8, 3

☐ Yes

☒ No

9 cannot be before 8.

First Person:

Given the directed graph above, is the following sequence a valid topological ordering? Explain your reasoning.

1, 7, 2, 4, 6, 3, 5, 8, 9

☒ Yes

☐ No

Question 11:

First Person:

Construct a valid topological ordering with 1 and 3 before 7.

Second Person:

Construct a valid topological ordering with 4 and 6 before 2.

Question 12:

Fun Bonus:

What is the total number of valid topological orderings for this graph?

374